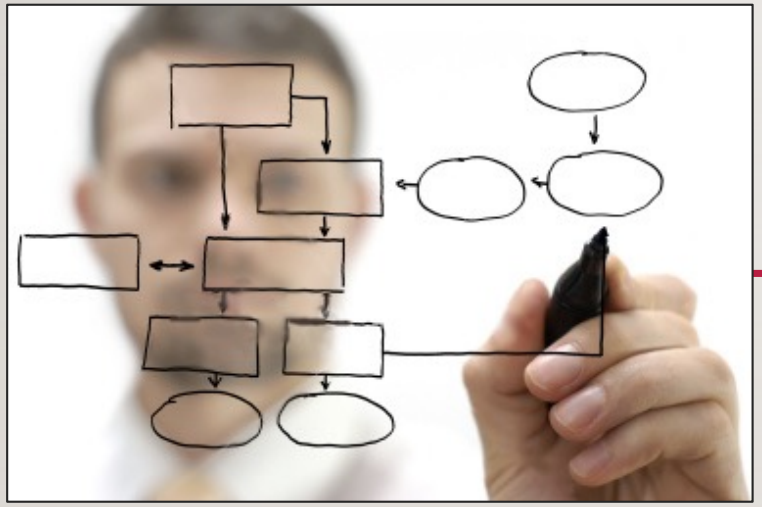
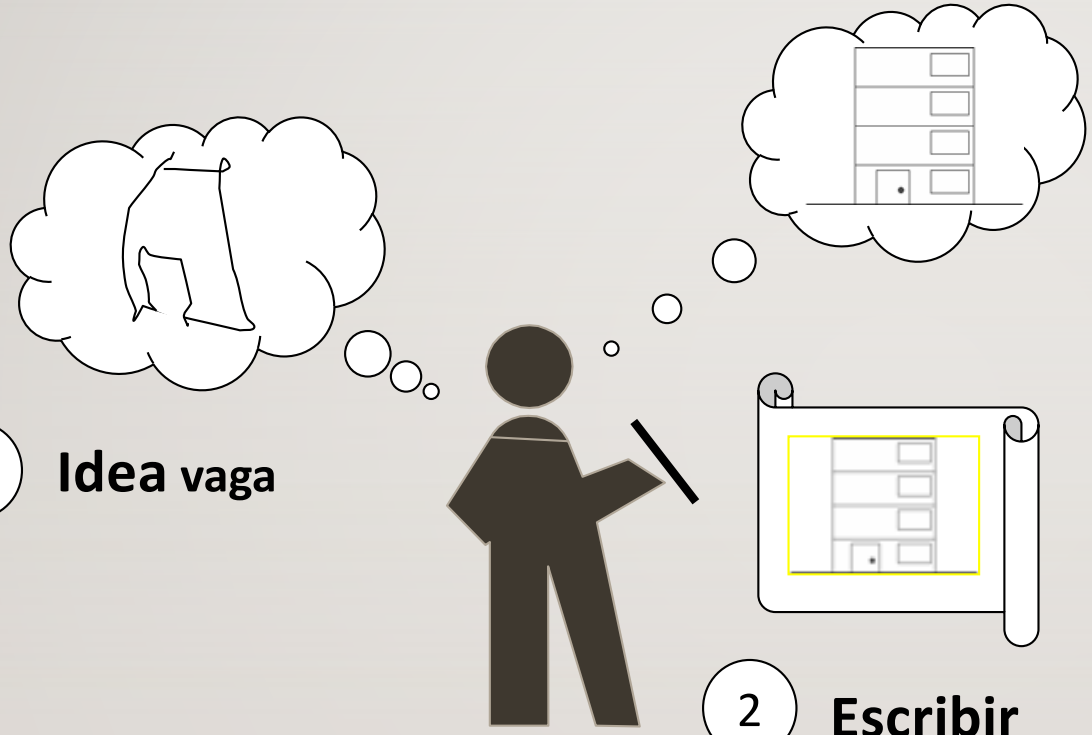
Diseño Conducido por Arquitectura



J. Andrés Díaz Pace

Diseño de Sistemas de Software

El Valor de Diseñar (y Documentar)



1 **Idea vaga**

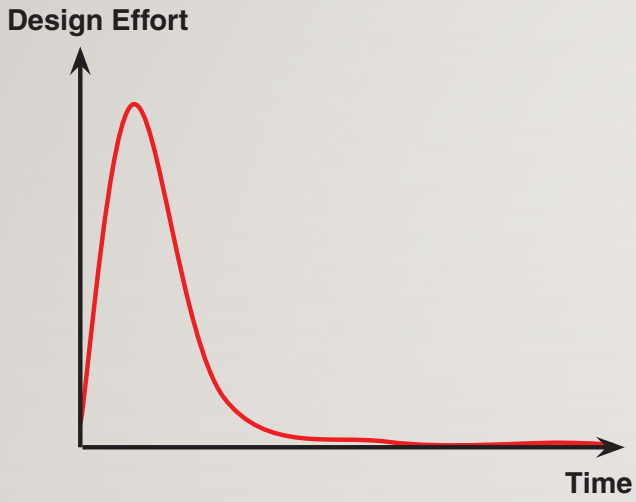
2 **Escribir**

3 **Idea enriquecida (conocimiento)**

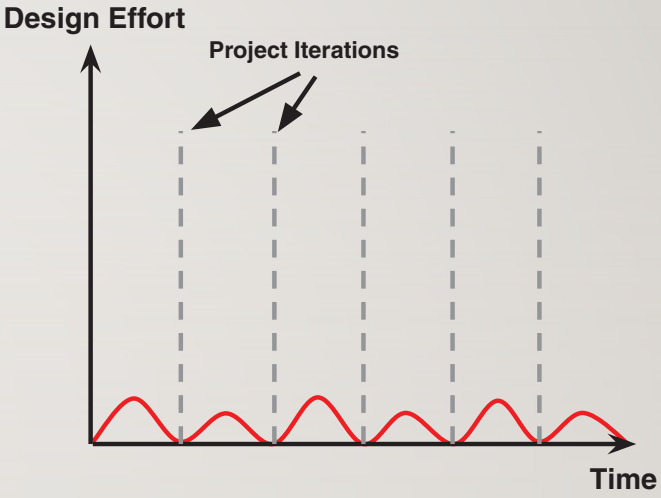
4 **Comunicar/Revisar**

FEEDBACK

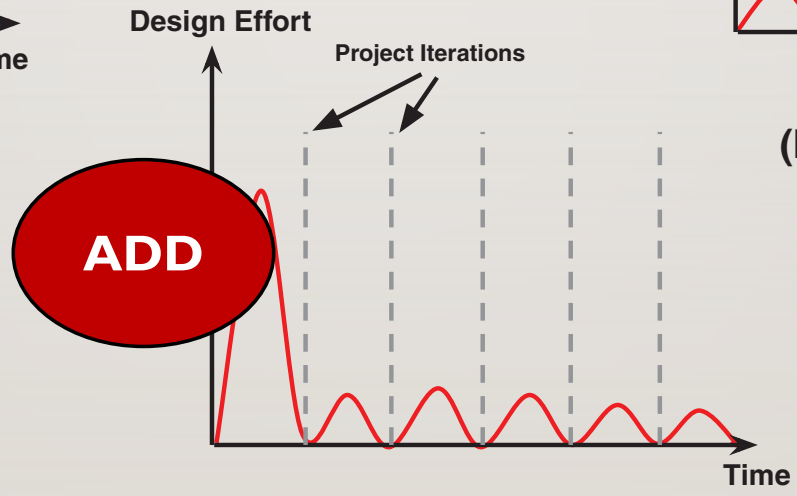
ENFOQUES PARA DISEÑO (CICLO DE VIDA)



(a) BDUF Approach



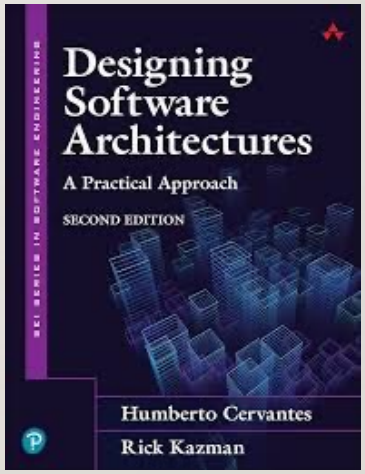
(b) Emergent Approach



(c) Iteration 0 Approach

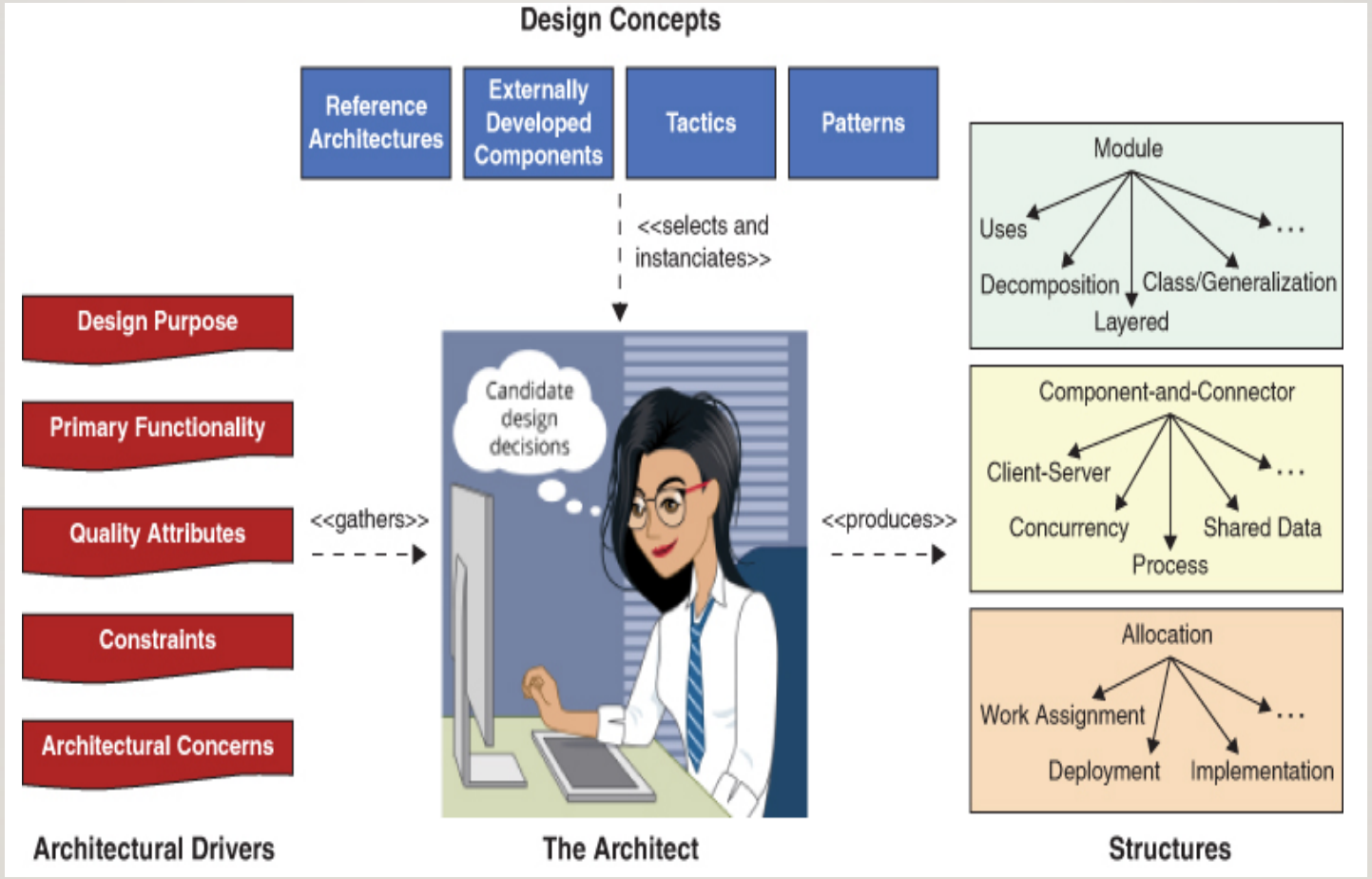
Attribute-Driven Design (ADD 3.0)

- El ADD fue desarrollado por el SEI a fin de aplicar atributos de calidad en la creación de arquitecturas de software
  - Sirve como guía para asegurarse que se están tomando bien las decisiones, y también para diseñar sistemáticamente
- El ADD sigue un **proceso de diseño recursivo basado en la descomposición** de un sistema (o elemento del sistema) por medio de tácticas y patrones que satisfacen los requerimientos claves

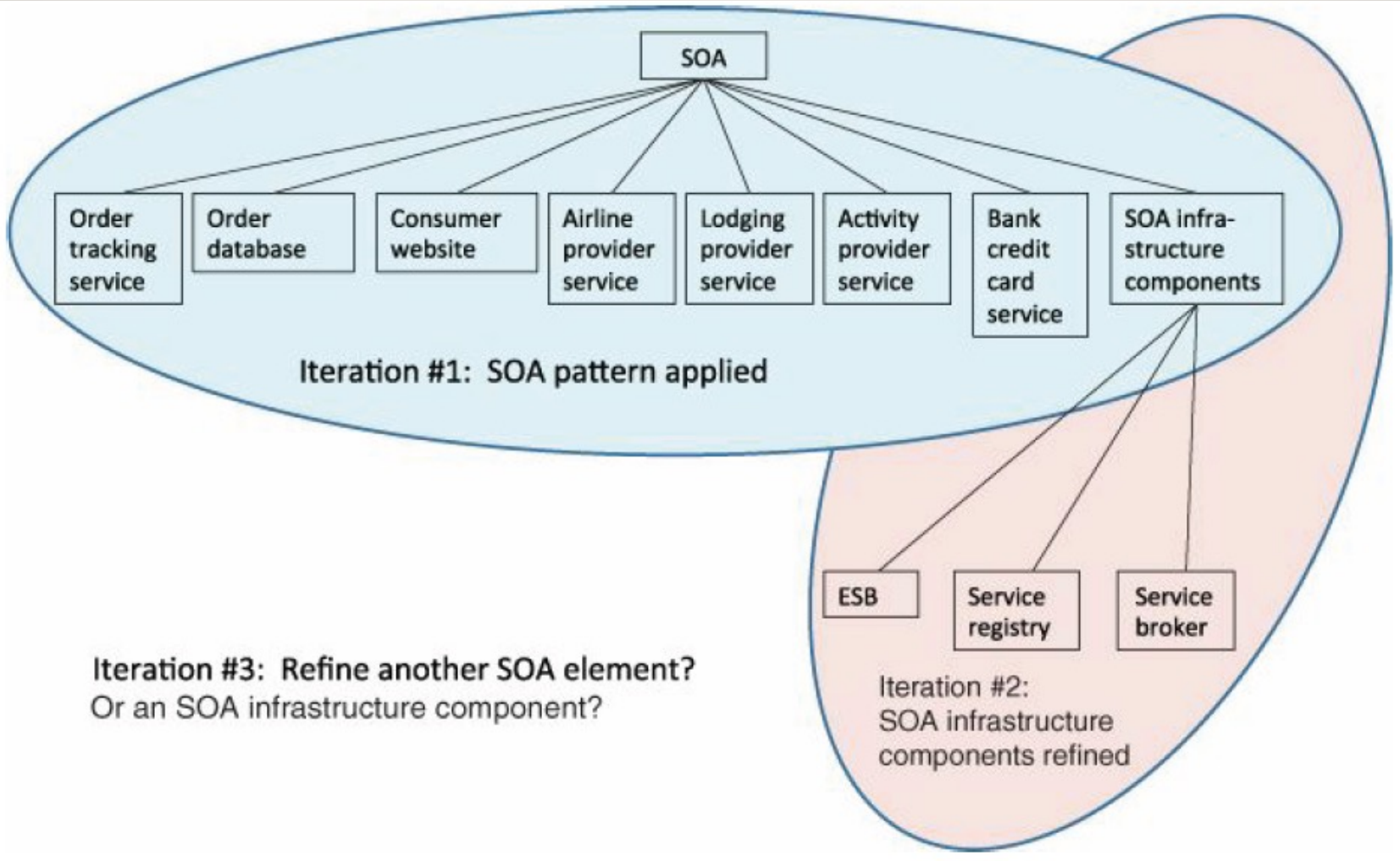


Software Engineering Institute
Carnegie Mellon

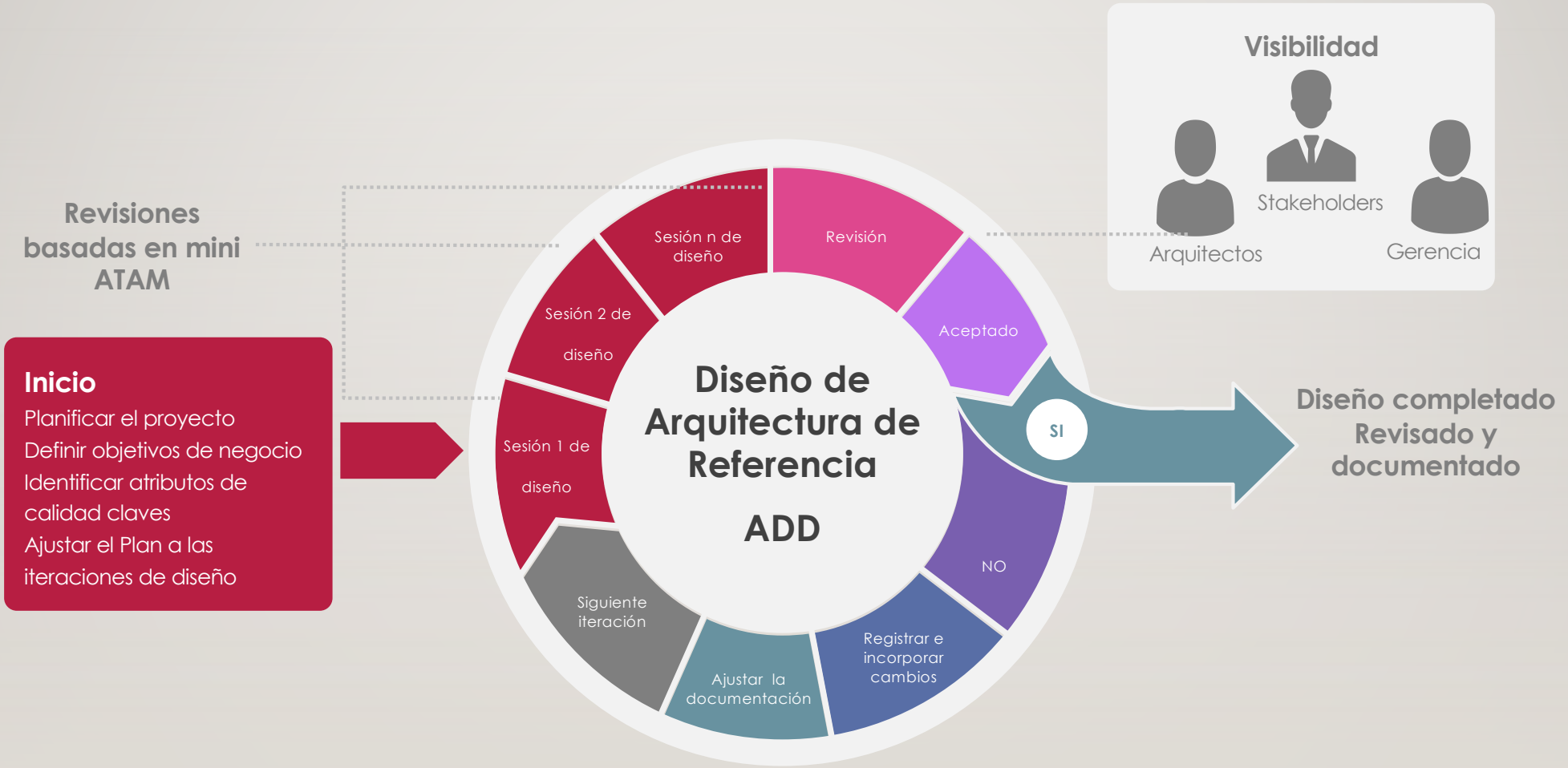
El contexto del Diseño



Descomposición en ADD



ITERACIONES DE DISEÑO (ADD)

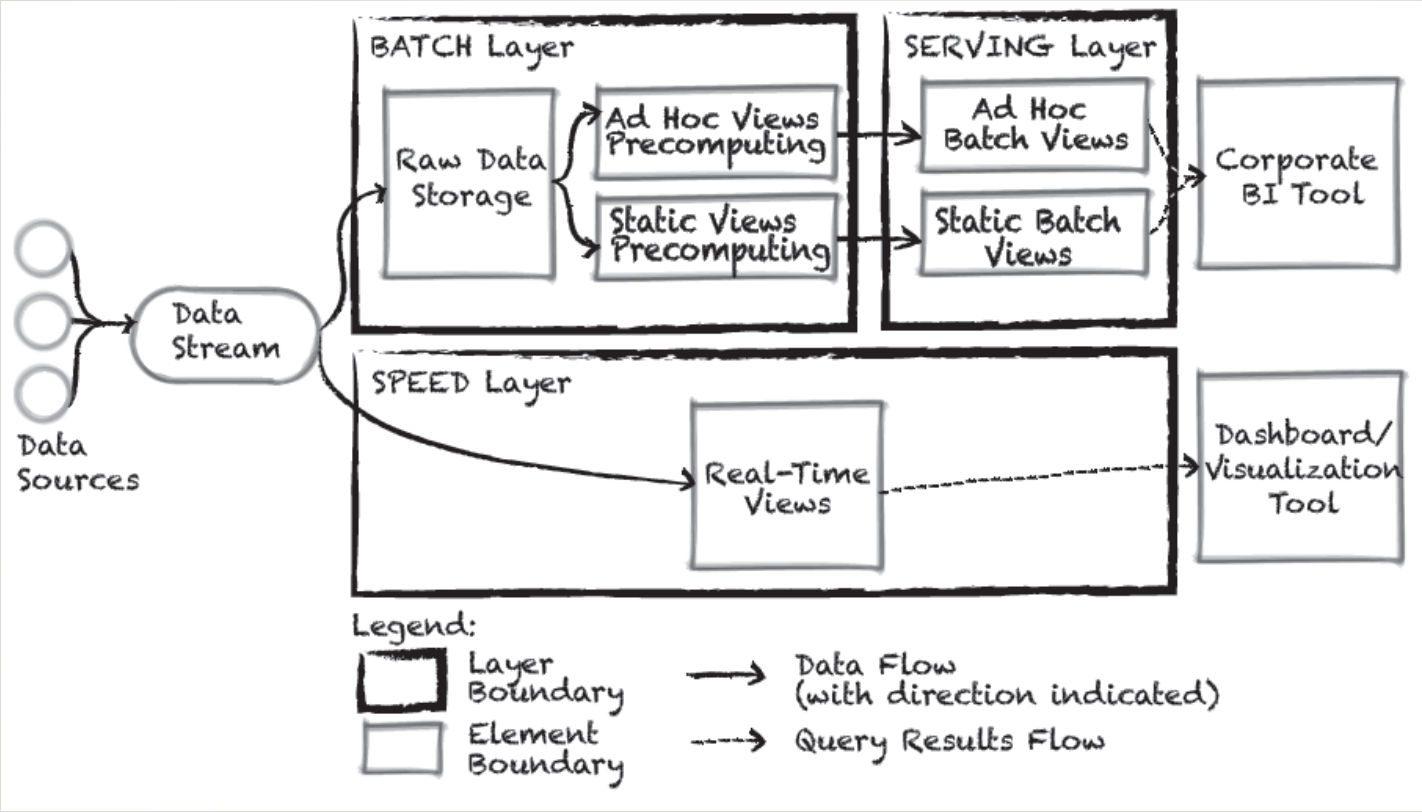


Sesiones de Diseño

- Un grupo (pequeño) de arquitectos, desarrolladores, analistas, y expertos, que toma ítems de un backlog de arquitectura (ej. JIRA)
  - Requerimientos funcionales claves
  - Escenarios de atributos de calidad (calidad de servicio, volumetría, etc.)

- Toma de decisiones y otras consideraciones
  - Pizarrón (sketch) + notas
  - No es estrictamente un "documento de arquitectura" (SAD)
  - La fase de consolidación de un SAD puede venir más tarde

Ejemplo de Sketch + Notas



Ejemplo de Sketch + Notas

Element	Responsibility
Data stream	This element collects data from all data sources in real time and dispatches it to both the batch layer and the speed layer for processing.
Batch layer	This layer is responsible for storing raw data and precomputing the batch views to be stored in the serving layer.
...	...

Driver	Design Decisions and Location	Rationale and Assumptions
QA-1	**Introduce concurrency** (tactic) in the `TimeServerConnector` and `FaultDetectionService`	Concurrency should be introduced to be able to receive and process several events (traps) simultaneously.
QA-2	Use of a **messaging** pattern through the introduction of a message queue in the communications layer	Although the use of a message queue may seem to go against the performance imposed by the scenario, a message queue was chosen because some implementations have high performance and, furthermore, this will be helpful to support QA-3.
...	...	...

Registro de Decisiones

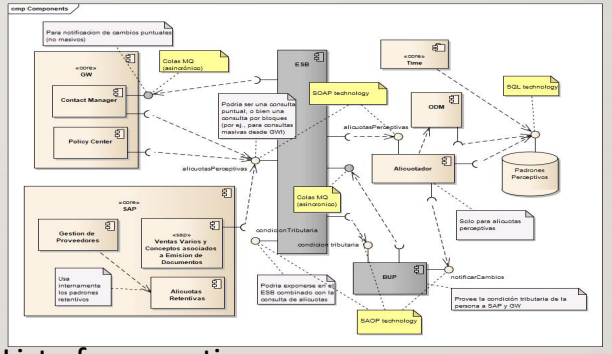
- Puede utilizarse un template (por ej., 1-2 páginas) o algún otro mecanismo
  - Lo importante es capturar las decisiones y sus consecuencias
  - Suele acompañarse con alguna vista de arquitectura
  - También se capturan los puntos "abiertos/pendientes" de la sesión (issues)

HLD Name

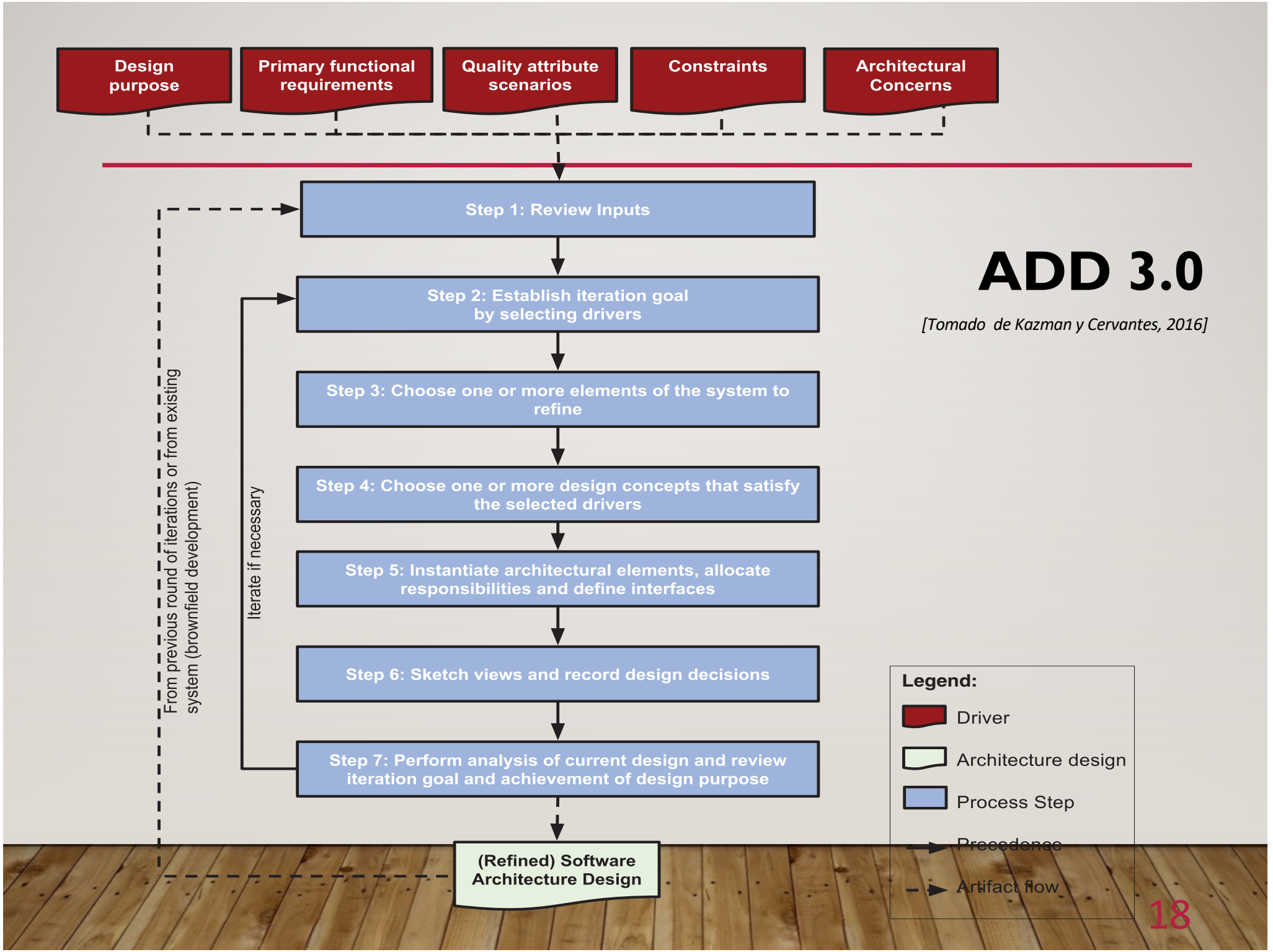
1. Description
- Domain & functional context
[Links to other related HLDs]

2. Solution Design

Design Decision	Rationale	Design Concerns
AD 1.1		
AD 1.2		
AD 1.3		



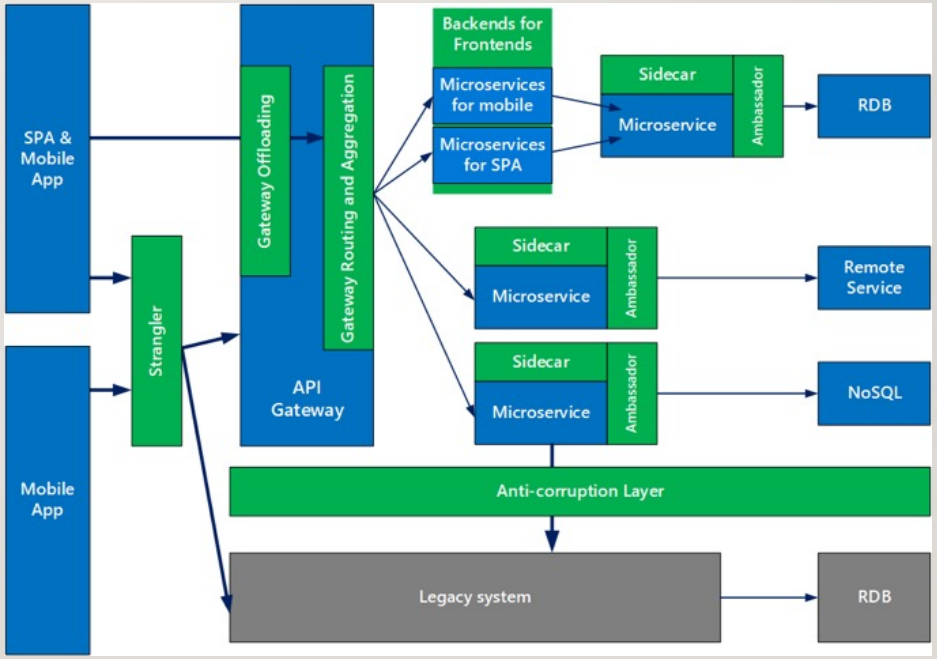
List of assumptions:
- *assumption 1*
- *assumption 2*


Design purpose
Primary functional requirements
Quality attribute scenarios
Constraints
Architectural Concerns
Step 1: Review Inputs
Step 2: Establish iteration goal by selecting drivers
Step 3: Choose one or more elements of the system to refine
Step 4: Choose one or more design concepts that satisfy the selected drivers
Step 5: Instantiate architectural elements, allocate responsibilities and define interfaces
Step 6: Sketch views and record design decisions
Step 7: Perform analysis of current design and review iteration goal and achievement of design purpose
(Refined) Software Architecture Design
From previous round of iterations or from existing system (brownfield development)
Iterate if necessary
ADD 3.0
[Tomado de Kazman y Cervantes, 2016]
Legend:
Driver
Architecture design
Process Step
Precedence
Artifact flow

Conceptos de Diseño

- Los sub-problemas atacados en una iteración generalmente pueden resolverse (re-)usando soluciones existentes

- Ejemplos de conceptos de diseño incluyen:
  - Arquitecturas de Referencia
  - Patrones
  - Tácticas
  - Componentes de terceros (ej., frameworks)
  - Familias de tecnologías

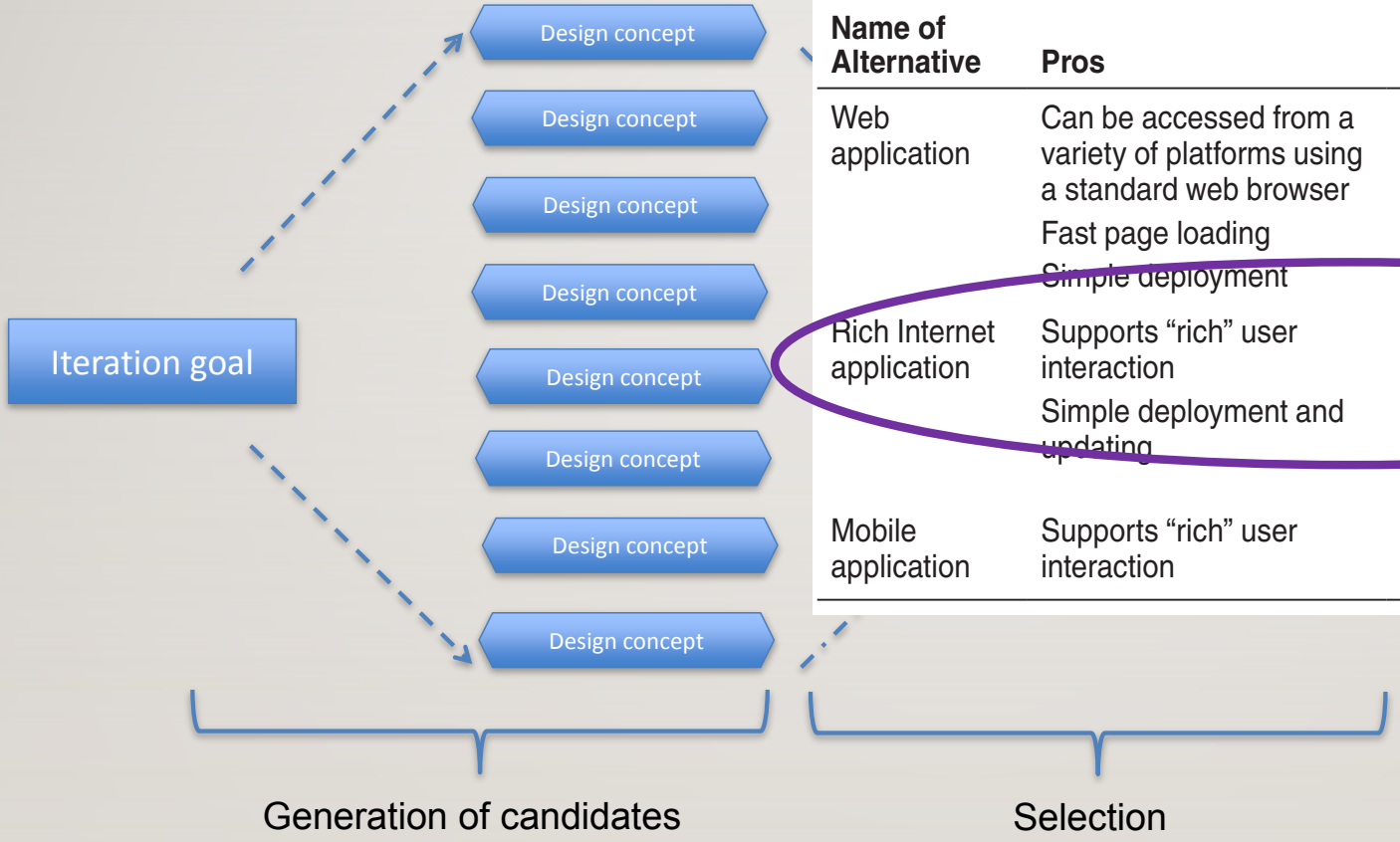


Objetivo de la iteración (Paso 2)

- **Architectural Drivers**: Un sub-conjunto de requerimientos que "dan forma" a la arquitectura
  - Requerimientos funcionales
  - Requerimientos de **atributos de calidad (escenarios)**
  - Restricciones

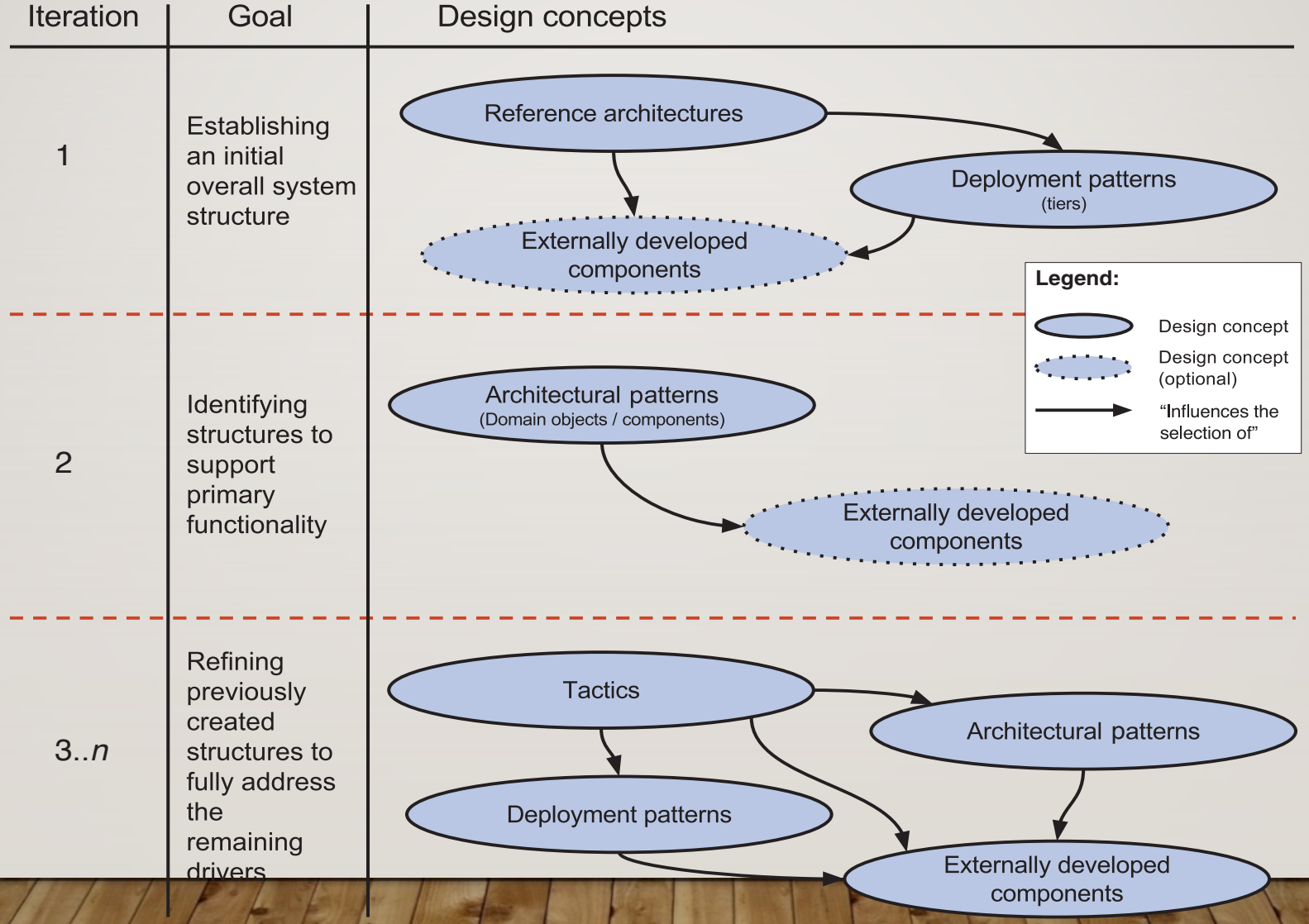
- Otros posibles drivers incluyen:
  - El **tipo de sistema** que se está diseñando (greenfield, brownfield)
  - **Objetivos** de diseño (diseño rápido para estimar, incremento de un sistema evolutivo, etc.)
  - **Concerns** (decisiones a tomar, aún cuando esté implícito)

Selección de conceptos de diseño (Paso 4)



Name of Alternative	Pros	Cons	Cost
Web application	Can be accessed from a variety of platforms using a standard web browser Fast page loading Simple deployment	Does not support "rich" interaction	Low
Rich Internet application	Supports "rich" user interaction Simple deployment and updating	Longer page loading times Requires a runtime environment to be installed on the client browser	Medium
Mobile application	Supports "rich" user interaction	Less portability Screen limitations	High

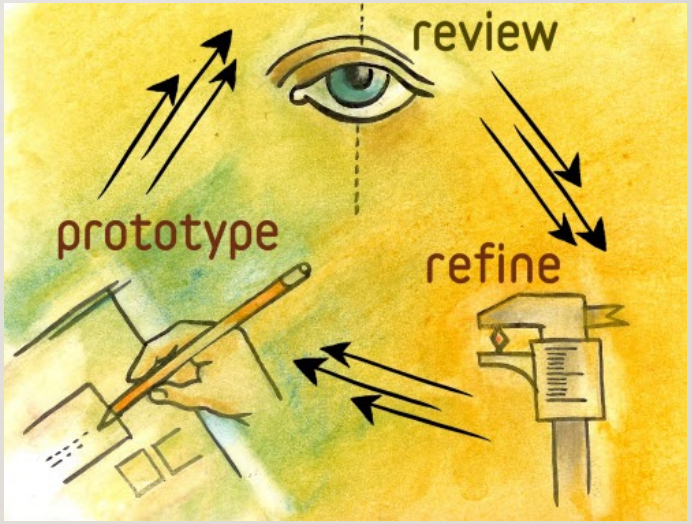
Selección en "greenfield" con dominio maduro



Selección en "greenfield" con dominio nuevo

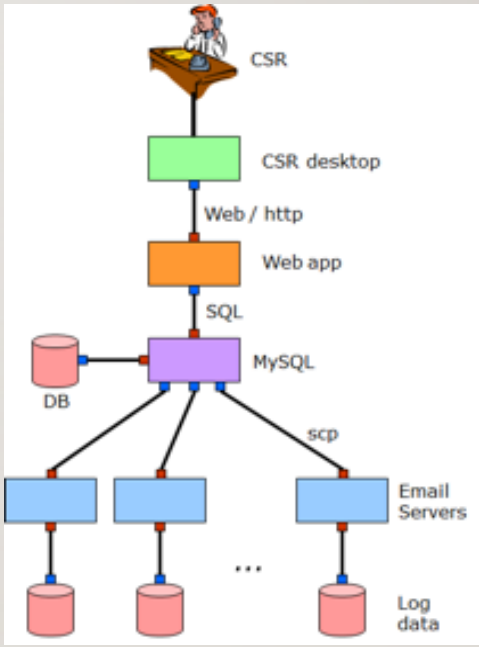
- Pueden no existir arquitecturas de referencia o patrones de despliegue

- Como alternativa, se pueden desarrollar "**prototipos**"
  - Que puedan irse evolucionando
  - Analizando atributos de calidad
  - Aplicando distintas tácticas



SELECCIÓN EN "BROWNFIELD"

- Requiere (inicialmente) un buen entendimiento de la arquitectura existente
  - Sistemas legados
- Generalmente involucra actividades de **refactoring (de diseño)**
  - Mejoras estructurales
  - Mejor soporte de ciertos atributos de calidad
  - **Evaluar impacto de realizar el refactoring**



Analizar diseño, y verificar cumplimiento del objetivo de la iteración (Paso 7)

- Al finalizar cada iteración, es recomendable verificar si las decisiones tomadas y el análisis realizado proveen suficiente evidencia para justificar que
  - Se cumplen con los escenarios de calidad deseados (tanto los de la iteración) como los atacados en iteraciones previas
  - Se satisfacen los lineamientos generales de arquitectura
  - Las decisiones de diseño guardan conformidad con la implementación
- Las revisiones pueden ser:
  - Manuales, basadas en herramientas (por ej., chequeos estáticos), análisis de logs, basadas en prototipos
- Son útiles para identificar **pendientes/issues/riesgos** hasta el momento en el proyecto

Caso de Estudio con ADD

- Aplicado a una arquitectura Big Data

- *Una empresa provee servicios contenidos y servicios online a millones de usuarios Web*
- *La empresa recolecta y analiza logs de datos masivos, que se generan de su infraestructura*
- *Distintos requerimientos de recolección (y cambiantes) por parte de distintos clientes*
- *Volumen y velocidad (near real-time)*
- *Es un contexto de diseño greenfield*

Casos de Uso

Use Case	Description
UC-1: Monitor online services	On-duty operations staff can monitor the current state of services and IT infrastructure (such as web server load, user activities, and errors) through a real-time operational dashboard, which enables them to quickly react to issues.
UC-2: Troubleshoot online service issues	Operations, support engineers, and developers can do troubleshooting and root-cause analysis on the latest collected logs by searching log patterns and filtering log messages.
UC-3: Provide management reports	Corporate users, such as IT and product managers, can see historical information through predefined (static) reports in a corporate BI (business intelligence) tool, such as those showing system load over time, product usage, service level agreement (SLA) violations, and quality of releases.

Use Case	Description
UC-4: Support data analytics	Data scientists and analysts can do ad hoc data analysis through SQL-like queries to find specific data patterns and correlations to improve infrastructure capacity planning and customer satisfaction.
UC-5: Anomaly detection	The operations team should be notified 24/7 about any unusual behavior of the system. To support this notification plan, the system shall implement real-time anomaly detection and alerting (future requirement).
UC-6: Provide security reports	Security analysts should be provided with the ability to investigate potential security and compliance issues by exploring audit log entries that include destination and source addresses, a time stamp, and user login information (future requirement).

Escenarios de Atributos de Calidad

- La vinculación entre escenarios y casos de uso ayuda luego a identificar la funcionalidad relevante para instanciar los patrones y tácticas de arquitectura

ID	Quality Attribute	Scenario	Associated Use Case
QA-1	Performance	The system shall collect up to 15,000 events/second from approximately 300 web servers.	UC-1, 2, 5
QA-2	Performance	The system shall automatically refresh the real-time monitoring dashboard for on-duty operations staff with < 1 min latency.	UC-1
QA-3	Performance	The system shall provide real-time search queries for emergency troubleshooting with < 10 seconds query execution time, for the last 2 weeks of data.	UC-2
QA-4	Performance	The system shall provide near-real-time static reports with per-minute aggregation for business users with < 15 min latency, < 5 seconds report load.	UC-3, 6
QA-5	Performance	The system shall provide ad hoc (i.e., non-predefined) SQL-like human-time queries for raw and aggregated historical data, with < 2 minutes query execution time. Results should be available for query in < 1 hour.	UC-4
QA-6	Scalability	The system shall store raw data for the last 2 weeks available for emergency troubleshooting (via full-text search through logs).	UC-2
QA-7	Scalability	The system shall store raw data for the last 60 days (approximately 1 TB of raw data per day, approximately 60 TB in total).	UC-4
QA-8	Scalability	The system shall store per-minute aggregated data for 1 year (approximately 40 TB) and per-hour aggregated data for 10 years (approximately 50 TB).	UC-3, 4, 6
QA-9	Extensibility	The system shall support adding new data sources by just updating a configuration, with no interruption of ongoing data collection.	UC-1, 2, 5
QA-10	Availability	The system shall continue operating with no downtime if any single node or component fails.	All use cases
QA-11	Deployability	The system deployment procedure shall be fully automated and support a number of environments: development, test, and production.	All use cases

Restricciones y Concerns

ID	Constraint
CON-1	The system shall be composed primarily of open source technologies (for cost reasons). For those components where the value/cost of using proprietary technology is much higher, proprietary technology may be used.
CON-2	The system shall use the corporate BI tool with a SQL interface for static reports (e.g., MicroStrategy, QlikView, Tableau).
CON-3	The system shall support two specific deployment environments: private cloud (with VMware vSphere Hypervisor) and public cloud (Amazon Web Services). Architecture and technology decisions should be made to keep deployment vendor as agnostic as possible.

ID	Concern
CRN-1	Establishing an initial overall structure as this is a greenfield system.
CRN-2	Leverage the team's knowledge of the Apache Big Data ecosystem.

PRIORIZACIÓN

- La priorización es importante para focalizar el esfuerzo y "forzar" tradeoffs

Scenario ID	Importance to Customer	Difficulty of Implementation According to Architect
QA-1	High	High
QA-2	High	Medium
QA-3	Medium	Medium
QA-4	High	High
QA-5	Medium	High
QA-6	Medium	Medium
QA-7	Medium	Medium
QA-8	High	Medium
QA-9	High	Medium
QA-10	High	Medium
QA-11	Medium	High

Conceptos de Diseño para Big Data

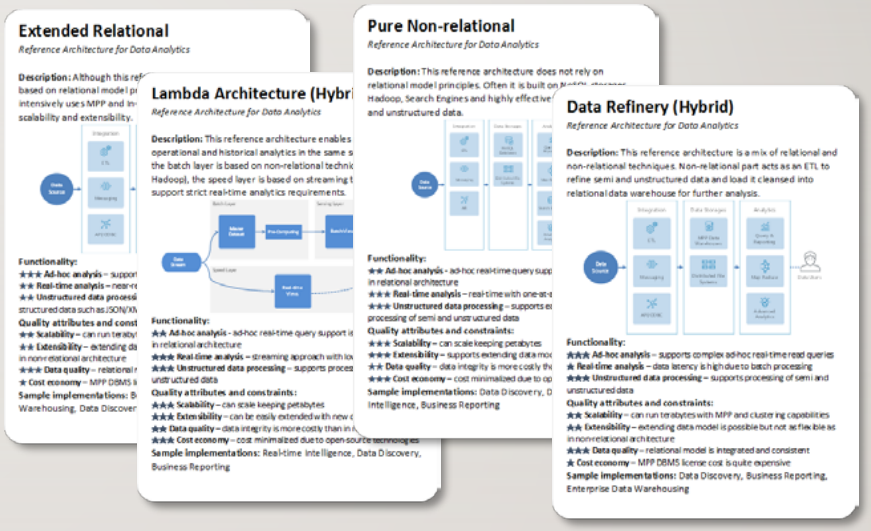
https://patterns.arcitura.com/big-data-patterns

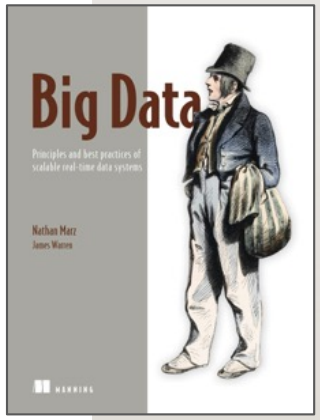
Catálogo de mecanismos y patterns

- Dispositivo de almacenamiento
- Motor de procesamiento
- Gestor de recursos
- Motor de transferencia de datos
- Motor de serialización
- Motor de compresión
- Motor de consultas
- Motor de analítica
- Motor de workflow
- Motor de coordinación
- Gestor de gobierno de datos
- Portal de productividad
- Motor de seguridad
- Gestor de cluster
- Motor de visualización
- …

Arcitura®

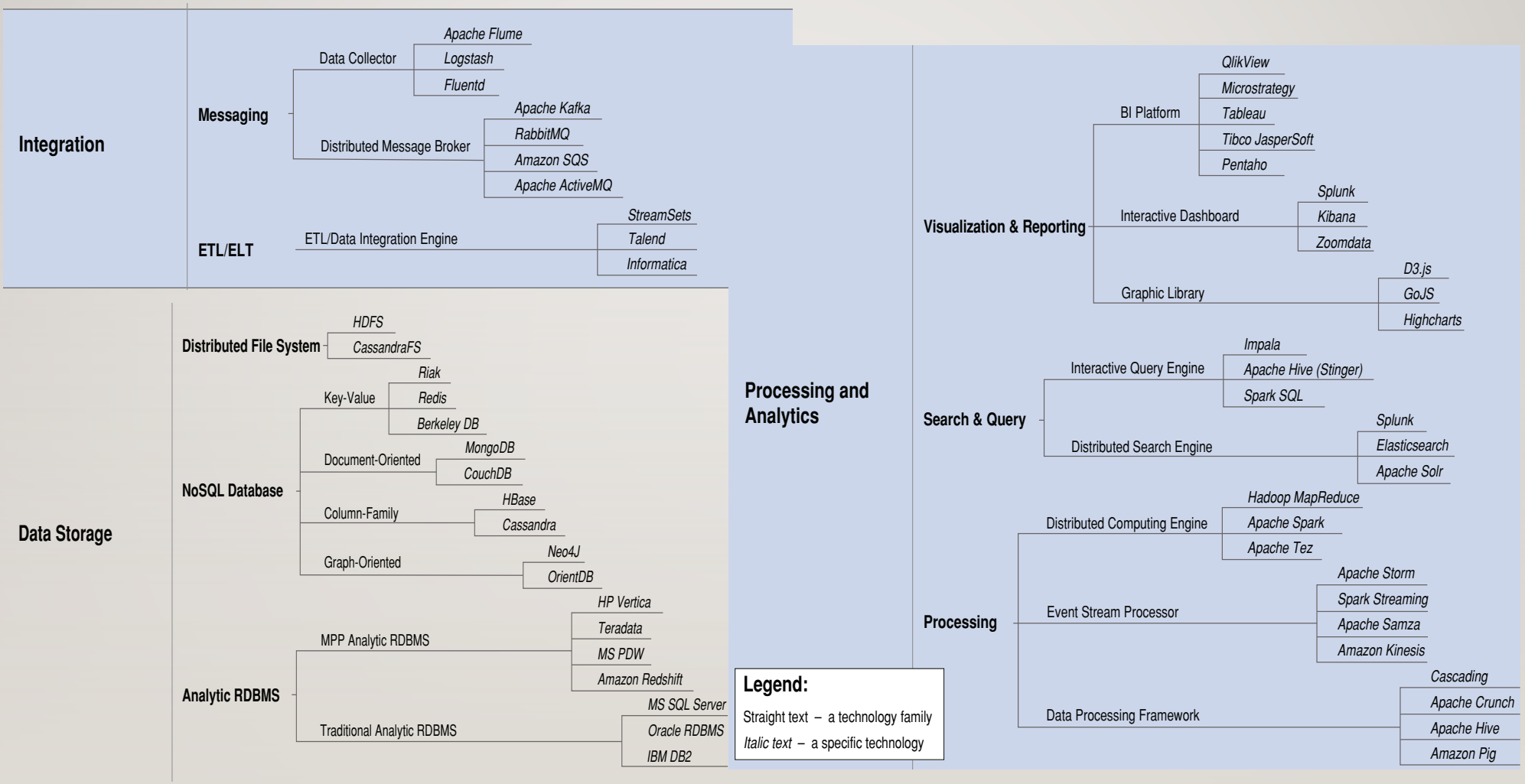
https://smartdecisionsgame.com/





http://lambda-architecture.net/

Relación con Tecnologías



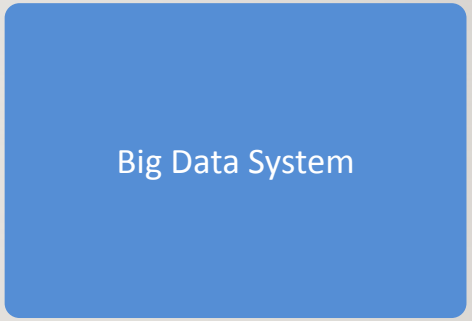
TP Especial: Ejercicio ADD

- Realizar un proceso de ADD para un caso de estudio, partiendo de un conjunto de requerimientos funcionales predefinidos, e identificando un conjunto de requerimientos de atributos de calidad drivers
- Aplicar TODO lo visto hasta el momento

ITERACIÓN 1: ARQUITECTURA DE REFERENCIA

- CON-1: Leverage open source technologies whenever applicable
- CON-2: Use corporate BI tool with SQL interface for static reports
- CON-3: Two deployment environments: private and public clouds
- QA-1, 2, 3, 4, 5: Performance
- QA-6, 7, 8: Scalability
- QA-9: Extensibility
- QA-10: Availability
- QA-11: Deployability

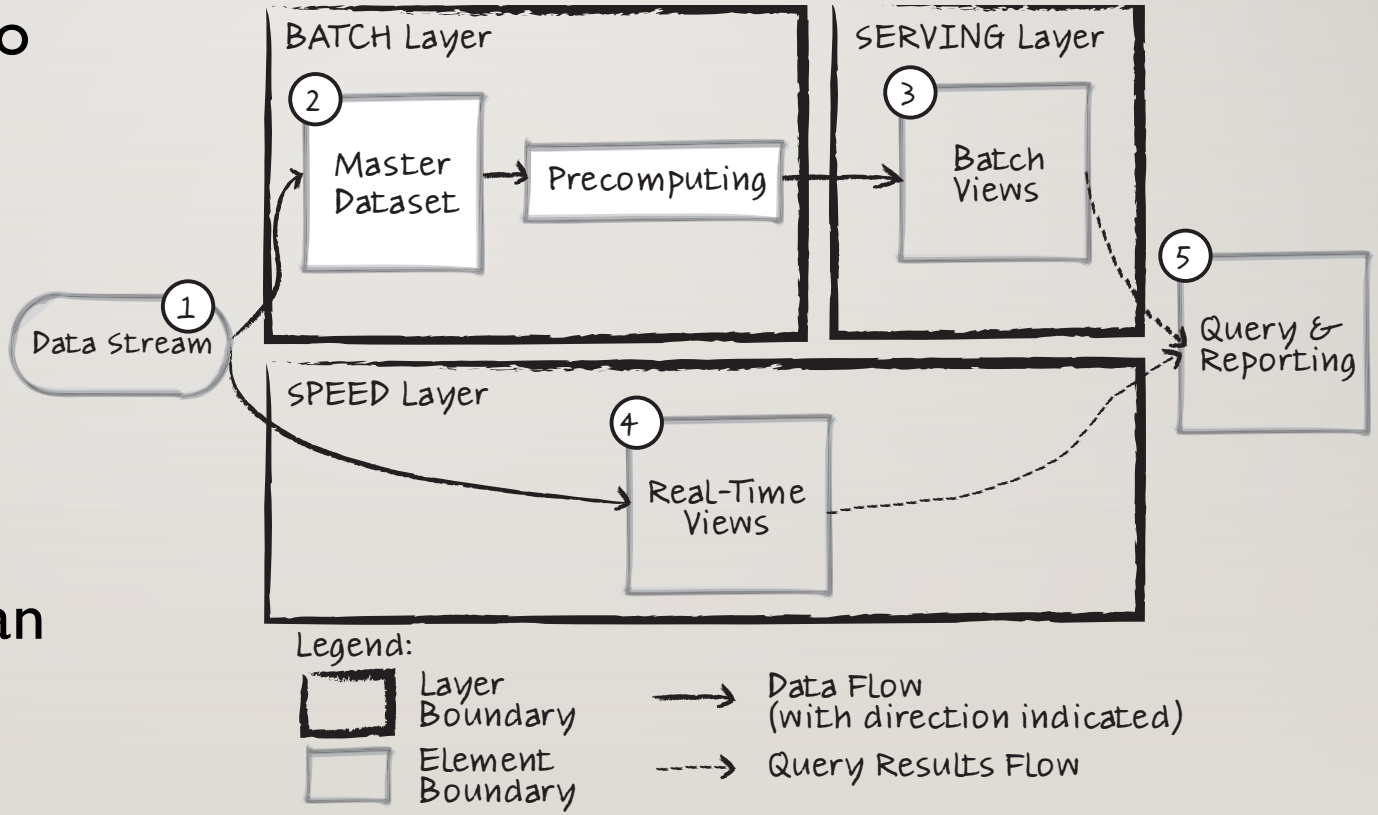
- Se deben analizar distintas opciones para arquitecturas de referencia para analítica. Para ello, se puede recurrir a:
  - Conocimiento codificado sobre Big Data
  - Información de proveedores
  - Experiencias personales



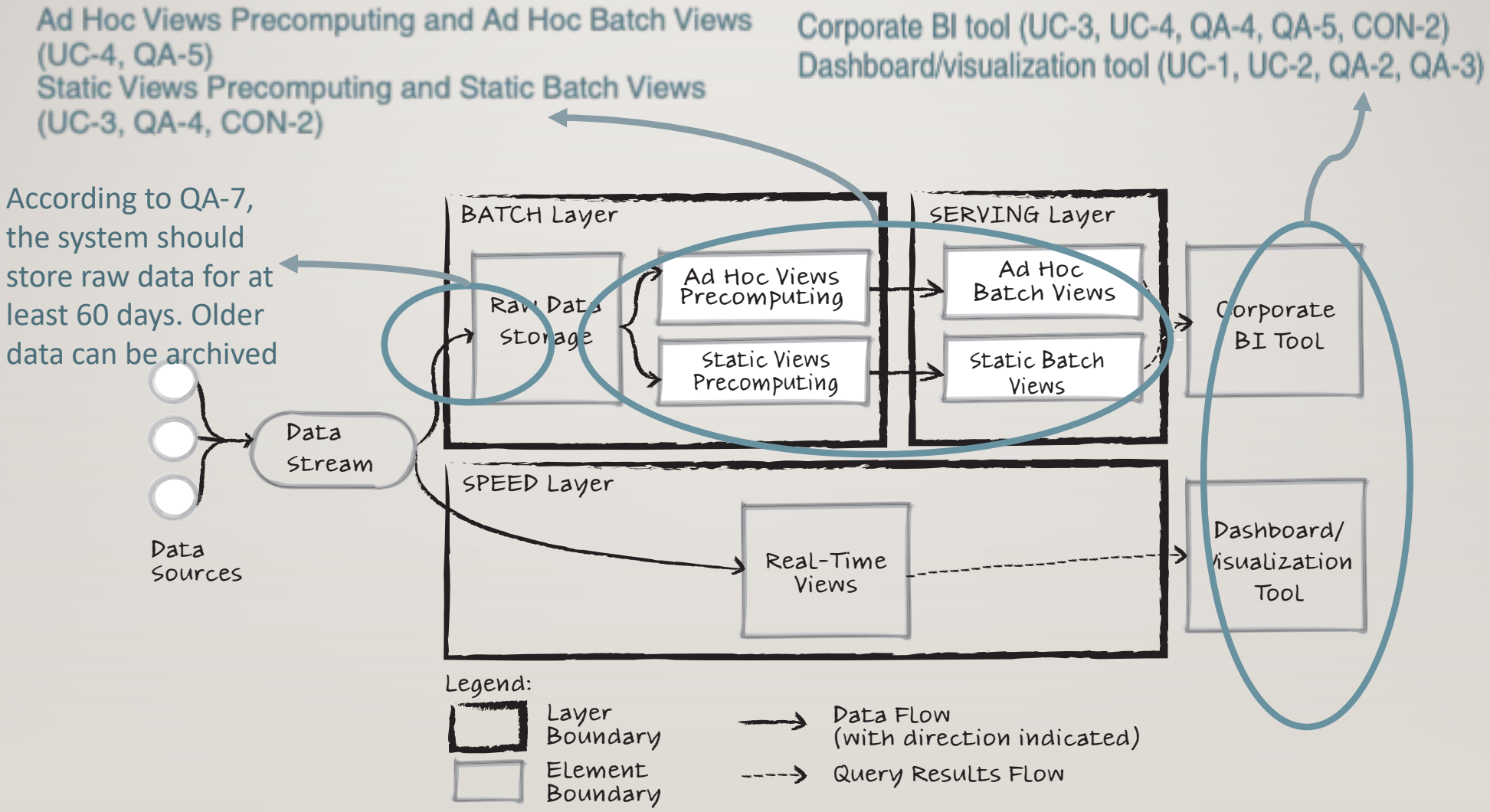
Ad-Hoc Analysis
Real-time Analysis
Unstructured data processing
Scalability
Cost Economy

Selección de Lambda

- Se debe justificar porqué Lambda es una alternativa razonable (y porqué se descartaron otras opciones)
- Considerar cómo "instanciar" Lambda para el proyecto
  - Información de casos de uso
- No se seleccionan tecnologías todavía



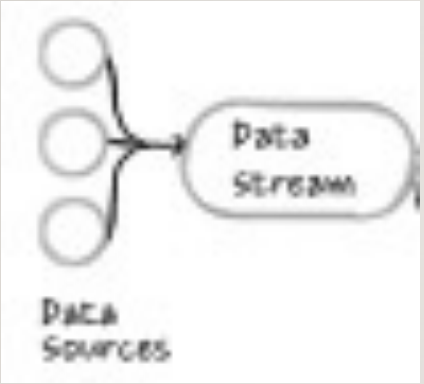
Instanciación de Lambda



ITERACIÓN 2: SELECCIÓN DE TECNOLOGÍA

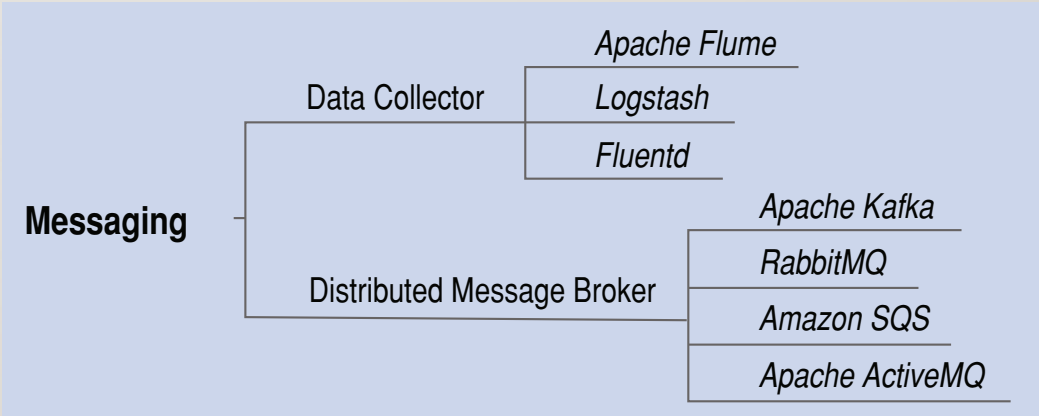
- La selección de tecnologías (y especialmente en Big Data) influencia la estructura arquitectónica
- Adicionalmente, una tecnología puede generar dependencias o incompatibilidades con otras tecnologías
- La noción de "familia de tecnologías" puede facilitar un análisis más agnóstico de implementaciones concretas
  - Algunas tecnologías dentro de la misma familia pueden ser intercambiables

ITERACIÓN 2A: DATA STREAM

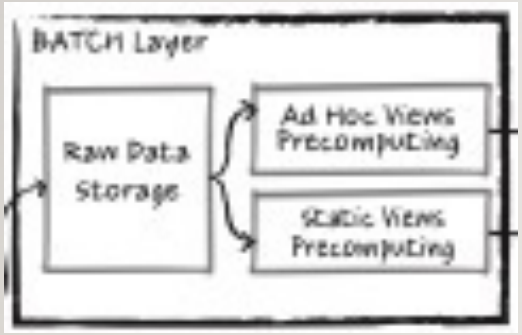


Design Decisions and Location	Rationale and Assumptions
Select the Data Collector family for the Data Stream element	Data Collector is a technology family (and an architectural pattern) that collects, aggregates, and transfers log data for later use. Usually Data Collector implementations offer out-of-the-box plug-ins for integrating with popular event sources and destinations. The destinations are the Raw Data Storage and Real-Time Views elements, which will also be addressed in this iteration.

- Drivers de la iteración:
  - Performance
  - Compatibility
  - Reliability
- Se descartaron alternativas:
  - ETL engine
  - Distributed Message Broker

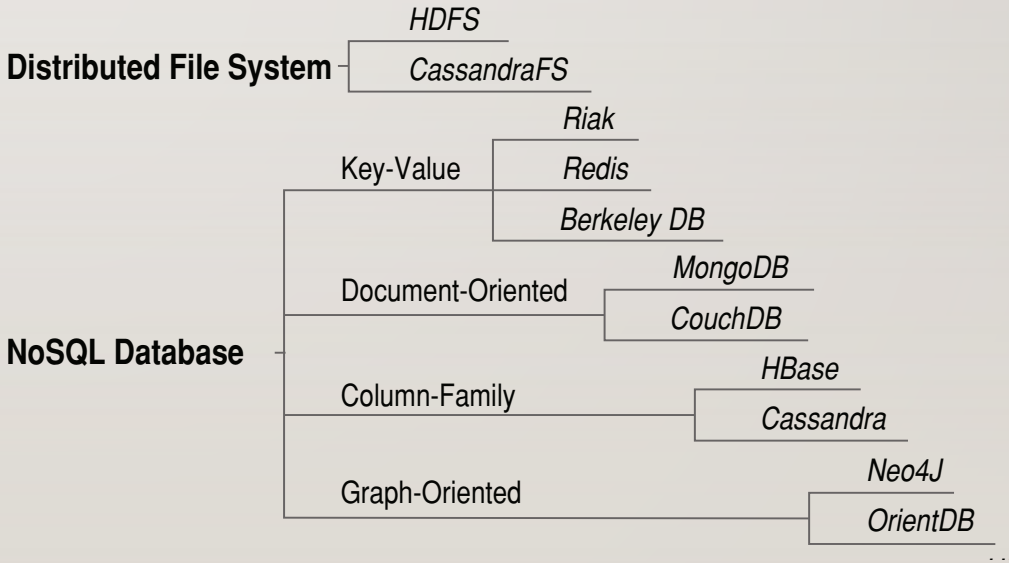


ITERACIÓN 2B: BATCH LAYER

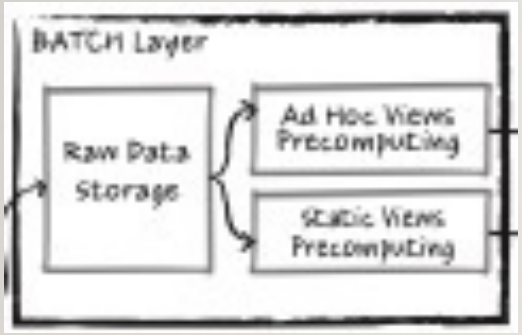


Design Decisions and Location	Rationale and Assumptions
Select the Distributed File System family for the Raw Data Storage element	According to the Lambda architecture principles, the Raw Data Storage element must be immutable. Thus new data should not modify existing data, but just be appended to the dataset. Data will be read in batch operations for transforming raw data to Batch Views. For these purposes, we can confidently choose a Distributed File System.

- Drivers de la iteración:
  - Scalability
  - Availability
- Se descartaron alternativas:
  - NoSQL database
  - Analytic RDBMS

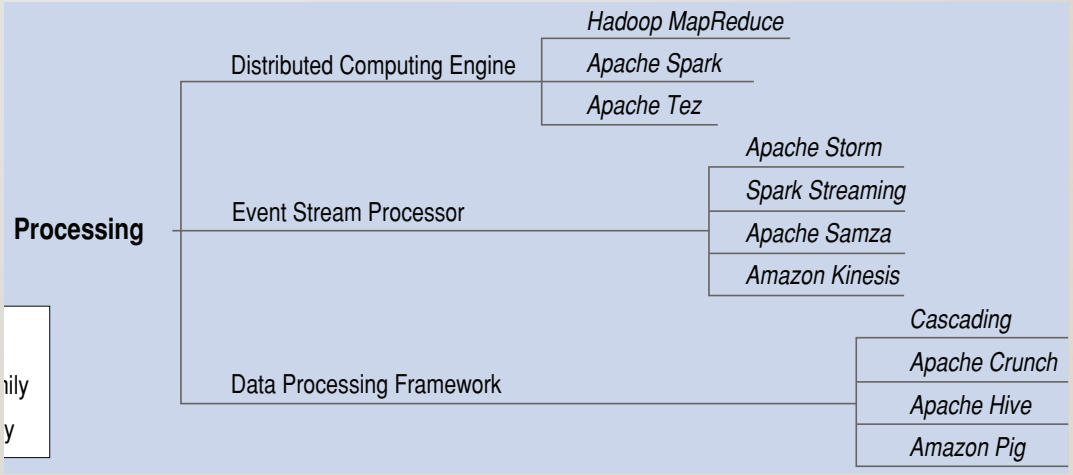


ITERACIÓN 2B: BATCH LAYER



Design Decisions and Location	Rationale and Assumptions
Use Data Processing Framework for the Views Precomputing elements	As we have already selected the Distributed File System family for Raw Data Storage and Batch Views, the next step is to choose a solution for data transformation from the Raw Data Storage to the format used in the Batch Views. The decision is to select Data Processing Framework as this technology family allows data processing pipelines to be created using abstractions that support faster development and better maintainability.

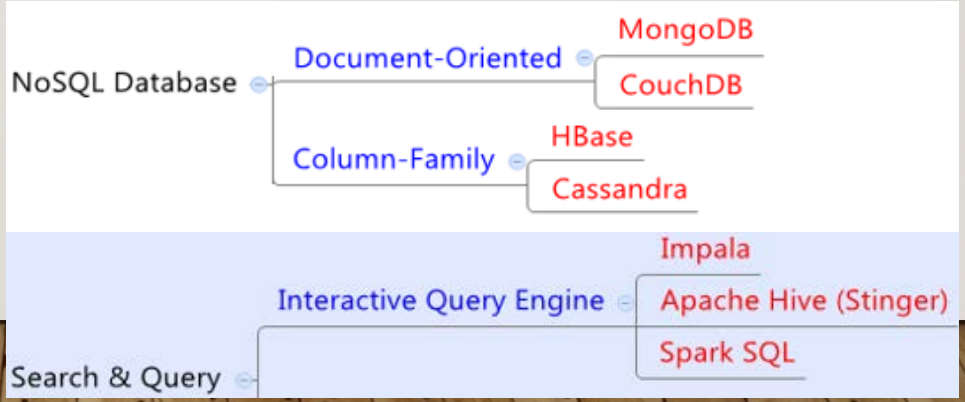
- Drivers de la iteración:
  - Scalability
  - Availability
- Se descartaron alternativas:
  - Distributed Computing engine
  - Event Stream processor



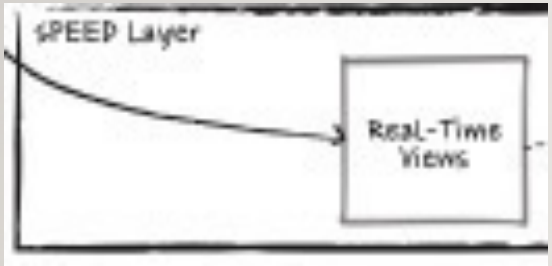
ITERACIÓN 2C: SERVING LAYER

Design Decisions and Location	Rationale and Assumptions
Select Interactive Query Engine family for both the Static and Ad Hoc Batch Views elements	As we stated in the previous iteration, the Batch Views element is refined into two elements, the Static and Ad Hoc Batch Views, to support two use cases: the generation of static reports (UC-3, 6) and the support for ad hoc querying (UC-4). The main design decision is to use the same technology family for both Static and Ad Hoc Batch Views—namely, the Interactive Query Engine. These engines allow analytic database capabilities over data stored in a Distributed File System (thus this technology family is also selected implicitly). If we select a technology that is fast enough, it can be used for both elements. The benefit of using a single technology family is that we do not need to have separate storage technologies for reporting and querying data.

- Drivers de la iteración:
  - Ad-hoc analysis
  - Performance
- Se descartaron alternativas:
  - NoSQL database
  - Analytic RDBMS

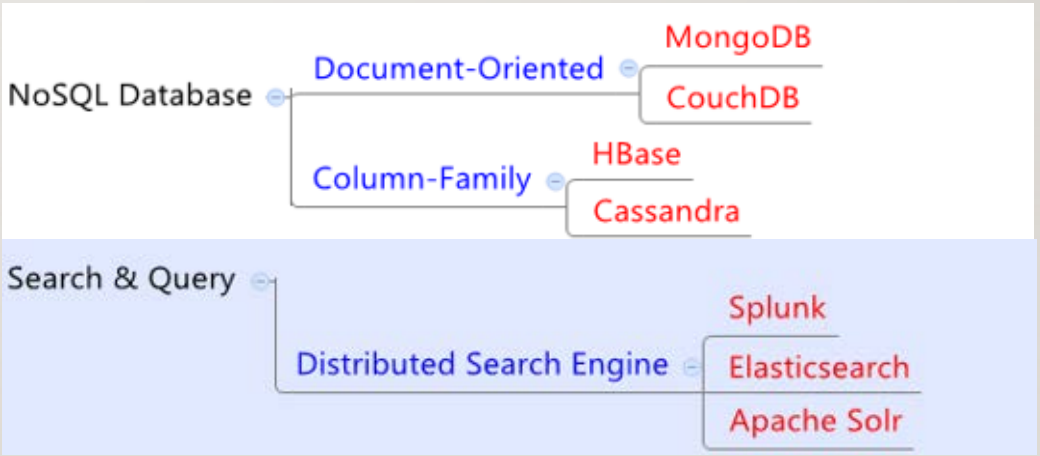


ITERACIÓN 2A: SPEED LAYER

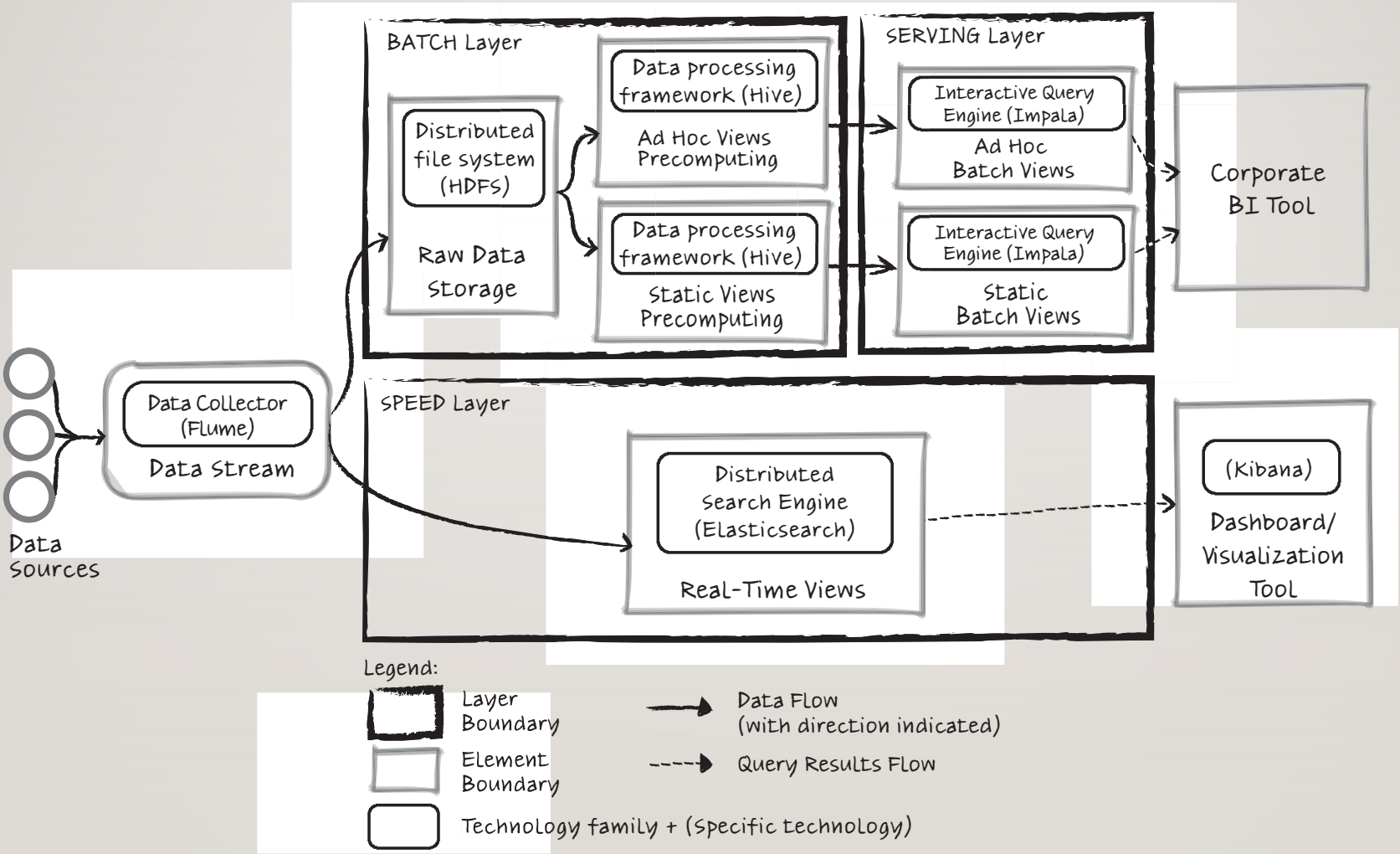


Design Decisions and Location	Rationale and Assumptions
Select Distributed Search Engine for the Real-Time Views element	The Real-Time Views element is responsible for full-text search over recent logs and for feeding an operational dashboard with real-time monitoring data (UC-1, UC-2). Distributed Search Engine is a technology family that serves just such purposes.

- Drivers de la iteración:
  - Ad-hoc analysis
  - Real-time analysis
- Se descartaron alternativas:
  - NoSQL dtabase
  - Analytic RDBMS
  - Distributed File System and Interactive Query engine



ITERACIÓN 2: INSTANCIACIÓN FINAL



¿YA TERMINÉ?

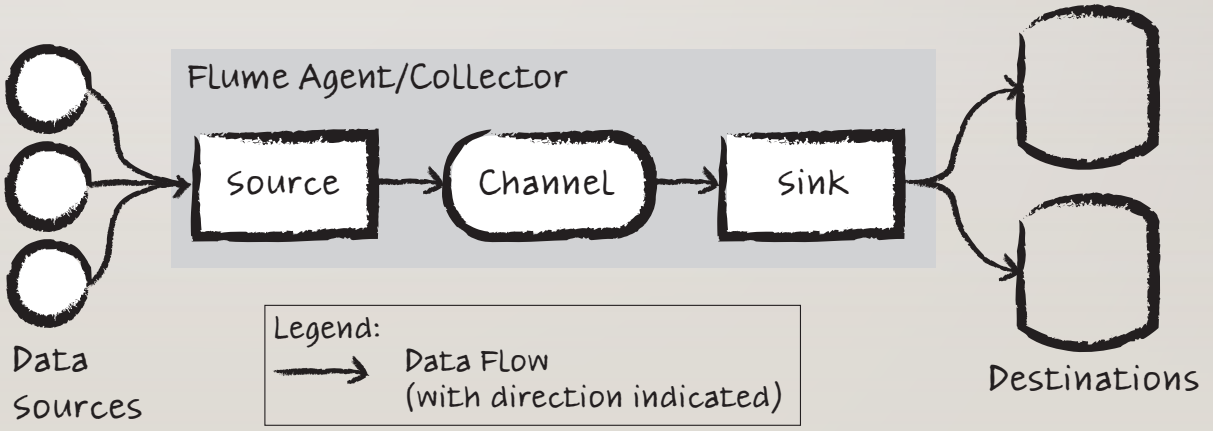
- La verificación de las decisiones tomadas en relación a los drivers indica:
  - **Satisfacción parcial** de algunos casos de uso y escenarios
  - La falta de ciertos detalles puede ocasionar **riesgos**

- Hasta el momento se ha hecho una descomposición "horizontal" Puede ser necesario realizar una descomposición "vertical" de algunos elementos
- Se debe continuar hasta que la evidencia de las decisiones y análisis realizados son "good enough" para el arquitecto (es decir, los riesgos han sido mitigados)

ITERACIÓN 3: REFINAMIENTO DATA STREAM

- No se tiene experiencia previa con el patrón Data Collector
  - Existen distintas variantes de implementación
  - Es critico para lograr una buena ingestión, y aprovechar las ventajas de la arquitectura Lambda

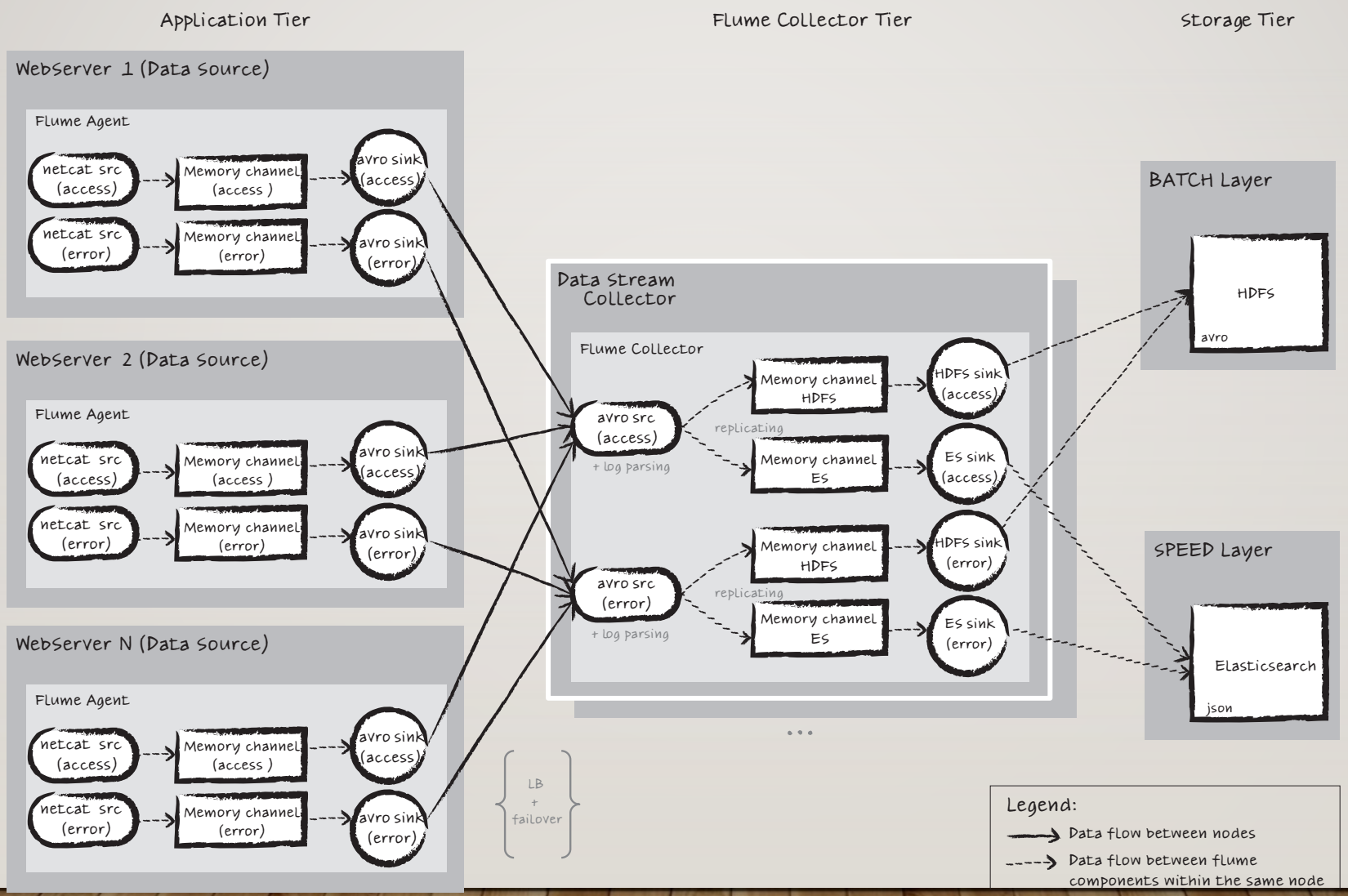
- QA-1 (Performance)
- QA-7 (Scalability)
- QA-9 (Extensibility)
- QA-10 (Availability)



ITERACIÓN 3: ALGUNAS DECISIONES

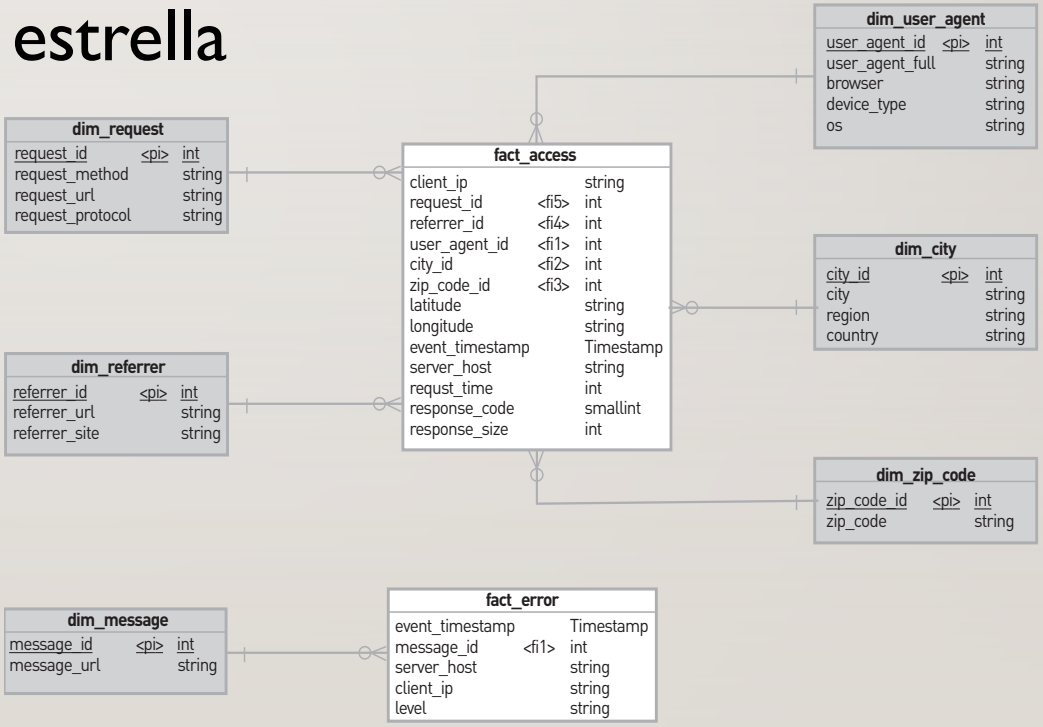
Design Decisions and Location	Rationale and Assumptions
Use Flume in agent/ collector configuration. Agents are co-located on the web servers, and the collector runs in the Data Stream element.	A Flume instance can run in two modes: as an agent (directly co-located in the data sources) or as a collector (which combines data streams from multiple agents and writes to destinations). From these two modes, Flume can be used in different configurations. The decision is to use Flume in both agent and collector configuration: The agents are co-located with the data sources and the Collector runs in the Data Stream element.
Introduce the tactic of "maintaining multiples copies of computations" by using a load-balanced, failover tiered configuration	Out of the possible topology alternatives, the selected one is a load-balanced and failover tiered topology based on performance (QA-1, 15,000 events/second) and availability (QA-10, no single point of failure) quality attribute scenarios.

ITERACIÓN 3: DIAGRAMA GENERADO



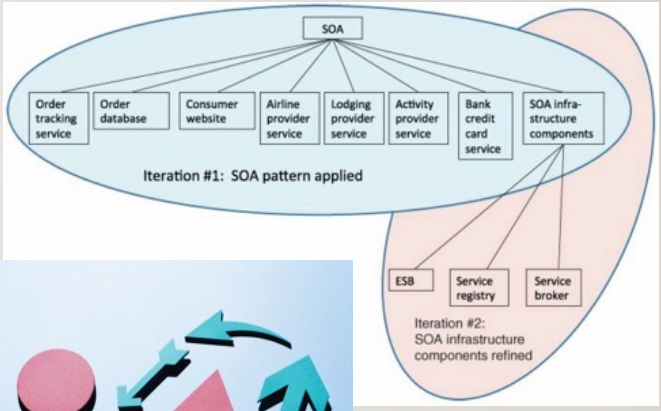
Iteración n: Más decisiones

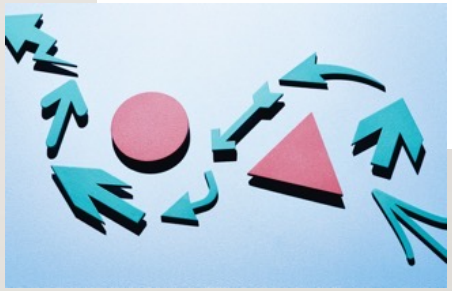
- Despliegue de la solución en Cloud Computing
- Selección de Parquet como formato de archivos para Impala en las Batch Views
- Utilización de esquema en estrella como modelo de datos
- …



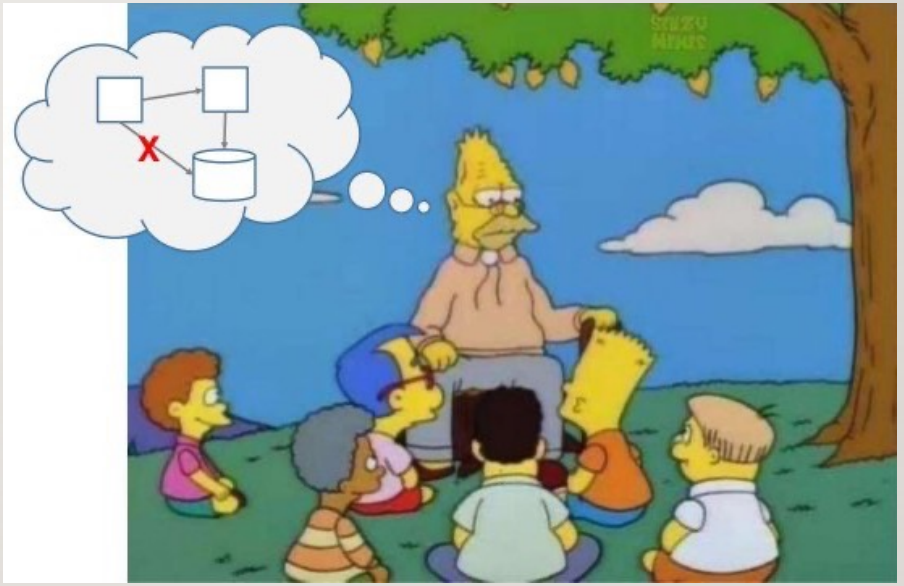
RESUMEN (DEL CASO)

- Se identificaron las diferentes entradas requeridas por el método
- Se partió de una arquitectura de referencia (particionamiento inicial)
- Se instanció la arquitectura en base a los requerimientos funcionales (horizontal)
- Se seleccionaron distintas tecnologías y patrones (horizontal y vertical)
- En el camino, se tomaron decisiones y se crearon diagramas





INTERCAMBIO



J. Andrés Díaz Pace

andres.diazpace@isistan.unicen.edu.ar